1.服务打包可执行文件

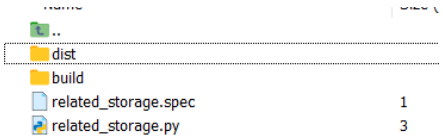
（1）python服务打包exe（wins）：

pyinstaller -F related_storage.py

（2）python服务打包（linux）：

1.pip install fastapi uvicorn pydantic pyinstaller等视情况而定

2.python3 -m PyInstaller -F xxx.py



3.输出目录：

4.linux服务器最好在宿主机打包环境，因为docker容器打包的执行文件可能与宿主机打包的文件系统不一致；

5.迁移服务器时要给文件权限：chmod +x related_storage（xxx）

6.运行可执行文件：到文件目录下->sudo ./related_storage（xxx）

2.pybind11插件形式（python集成到c++）

（1）架构：

project │ ├─ cpp │ └─ main.cpp │ ├─ python │ └─ associate_storage.py │ ├─ CMakeLists.txt

（2）cmake编译

cmake_minimum_required(VERSION 3.15)

project(pybind_demo)

find_package(pybind11 REQUIRED)

add_executable(main cpp/main.cpp)

target_link_libraries(main PRIVATE pybind11::embed)

（3）运行流程

mkdir build cd build cmake .. make

进入build目录：./main

3.基于pybind11编译py文件为.so/.dll共享库（python集成到c++）

1.把原来的python fastapi封装为函数py，如related_storage.py；

2.写一个c++.cpp来插件封装，注意：

```
py::module algo = py::module::import("related_storage");
```

里的import("related_storage")一定是你的py文件名！

3.CMake 构建 DLL（CMakeLists.txt）

如：

```
cmake_minimum_required(VERSION 3.15)

project(storage_plugin)

set(CMAKE_CXX_STANDARD 17)

find_package(pybind11 REQUIRED)

add_library(related_storage SHARED plugin.cpp)

target_link_libraries(related_storage PRIVATE pybind11::embed)
```

4.Windows编译

```
mkdir build
cd build
cmake ..
cmake --build . --config Release
```

（注：cmke ..可能会遇到**pybind11 没被 CMake 找到**的情况，这时候运行：

```
python -m pybind11 --cmakedir
```

会得到类似路径：

```
: C:\Users\xxx\AppData\Local\Programs\Python\Python311\Lib\site-
packages\pybind11\share\cmake\pybind11
```

然后重新执行：

```
cmake .. -Dpybind11_DIR="上面的路径"
```

)

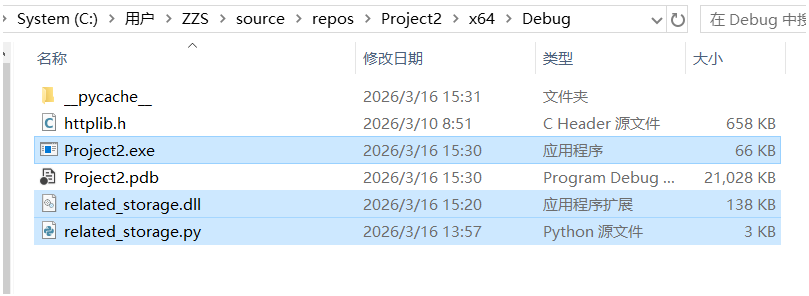
5.最后得到：

build	2026/3/16 15:20	文件夹	
CMakeLists.txt	2026/3/16 15:06	文本文档	1 KB
plugin.cpp	2026/3/16 15:19	C++ Source	1 KB
related_storage.py	2026/3/16 13:57	Python 源文件	3 KB

pybind11_dll > build > Release

名称	修改日期	类型	大小
related_storage.dll	2026/3/16 15:20	应用程序扩展	138 KB
related_storage.exp	2026/3/16 15:20	Exports Library F...	1 KB
related_storage.lib	2026/3/16 15:12	Object File Library	2 KB

要测试的话把py、dll文件放在c++项目里exe文件的同目录下：



System (C:) > 用户 > ZZS > source > repos > Project2 > x64 > Debug

名称	修改日期	类型	大小
__pycache__	2026/3/16 15:31	文件夹	
httplib.h	2026/3/10 8:51	C Header 源文件	658 KB
Project2.exe	2026/3/16 15:30	应用程序	66 KB
Project2.pdb	2026/3/16 15:30	Program Debug ...	21,028 KB
related_storage.dll	2026/3/16 15:20	应用程序扩展	138 KB
related_storage.py	2026/3/16 13:57	Python 源文件	3 KB

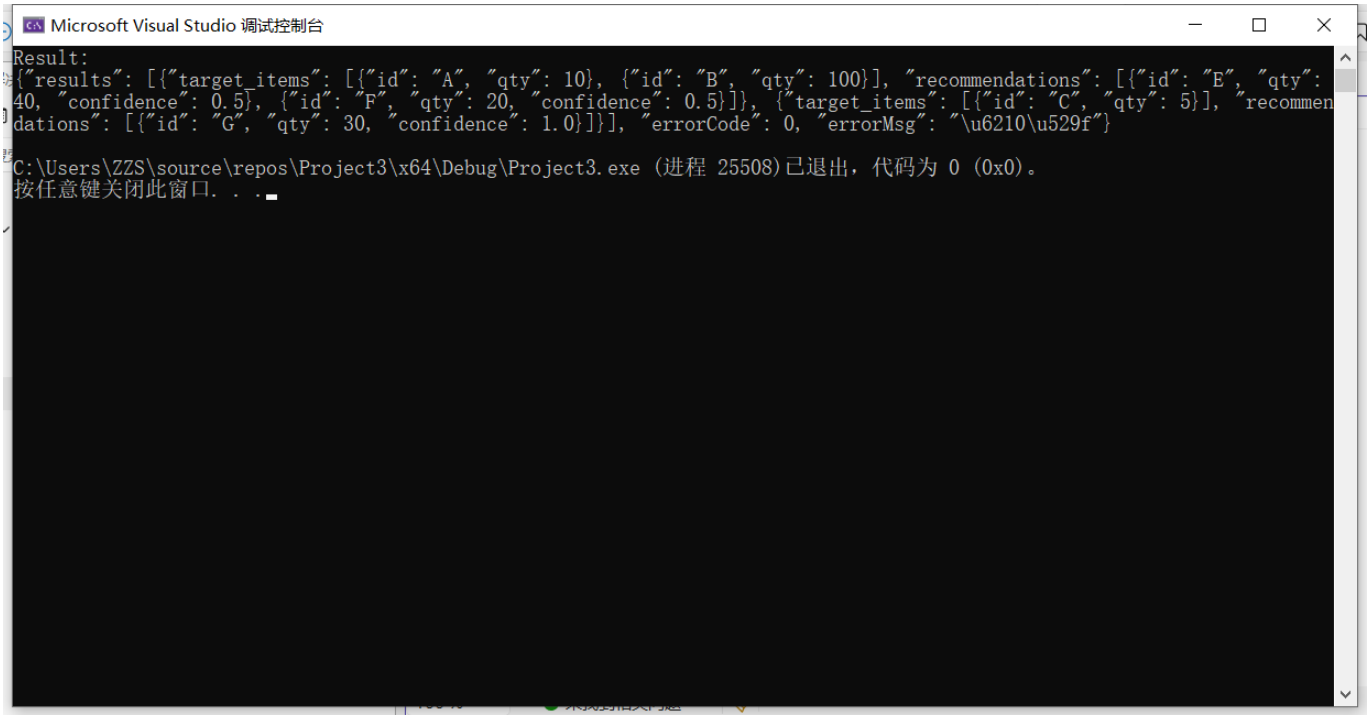
写一个测试cpp，如：

```
#include <windows.h>
#include <iostream>

typedef const char* (*AlgoFunc)(const char*);

int main()
{
    HMODULE lib = LoadLibraryA("related_storage.dll");

    if (!lib)
    {
        std::cout << "DLL load failed\n";
        return -1;
    }

    AlgoFunc func = (AlgoFunc)GetProcAddress(lib, "related_storage");

    if (!func)
    {
        std::cout << "function not found\n";
        return -1;
    }

    const char* json = R"(
{
  "target_items":[
    [
      {"id":"A","qty":10}
    ]
  ],
  "history_records":[],
  "min_support":1,
  "max_recommendations":3,
  "time_decay":0
}
)";

    const char* result = func(json);

    std::cout << "Result:\n" << result << std::endl;

    return 0;
}
```

在 VS 里按：Ctrl + Shift + B；

重新编译，然后运行：Projectx.exe

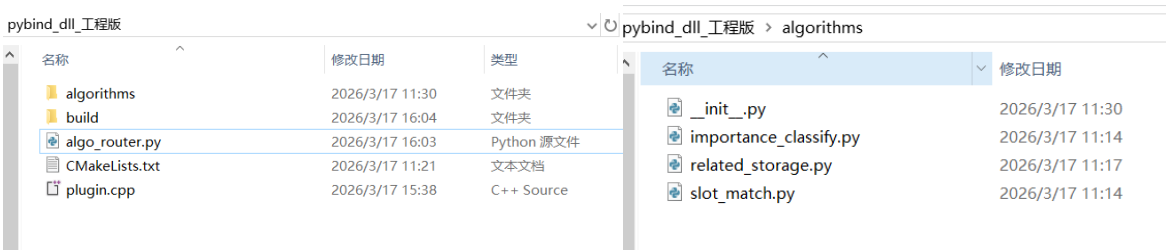
运行成功：



4.多py算法集成c++dll插件工程

1.结构明确

最好把所有封装的py放入一个文件夹下用于管理，另外需要一个管理c++调用py的类似路由的algo_router.py这样一个py



2.统一插件函数风格

因为要在c++里直接以AlgoFunc func =(AlgoFunc)GetProcAddress(lib, "run_algo");这样的形式调用，只需要传函数名即可，所以需要所有py函数的**接收、返回**格式统一

algo_router.py | related_storage.py × | slot_match.p

C: > Users > ZZS > Desktop > bky > pybind_dll_工程版 > algorithms

```python
def recommend(data):
    }


# ------------------------
# 统一插件入口
# ------------------------
def run(data):

    try:

        return recommend(data)

    except Exception as e:

        return {
            "results": [],
            "errorCode": 500,
            "errorMsg": str(e)
        }
```

3.按之前3里单个py服务的dll插件化一步步来cmake、生成dll

4.把dll文件、py文件、algorithm目录放入与test.exe同级目录下

:) > 用户 > ZZS > source > repos > pybind11_工程版 > x64 > Debug

名称	修改日期	类型
__pycache__	2026/3/17 16:04	文件夹
algorithms	2026/3/17 15:44	文件夹
algo_router.dll	2026/3/17 16:04	应用程序
algo_router.py	2026/3/17 16:03	Python
pybind11_工程版.exe	2026/3/17 16:06	应用程序
pybind11_工程版.pdb	2026/3/17 16:06	Progran

5.测试多个算法调用

在c++测试里：可直接输入算法名、json对象进行测试

```cpp
)";

    std::cout << "\n=== slot_match ===\n";
    std::cout << func("slot_match", json2) << std::endl;
```

因为在algo_router.py里：已经定义了统一的run（）函数来接受请求

```python
import importlib
import json
def run(algo_name, data):

    try:
        module = importlib.import_module(
            f"algorithms.{algo_name}"
        )

        result= module.run(data)
        # ✅ 强制 UTF-8 安全
        return json.loads(json.dumps(result, ensure_ascii=False))

    except Exception as e:

        return {
            "errorCode": 500,
            "errorMsg": str(e)
        }
```

结果如下：

Microsoft Visual Studio 调试控制台

```
=== related_storage ===
{"results": [{"target_items": [{"id": "A", "qty": 10}, {"id": "B", "qty": 100}], "recommendations": [{"id": "E", "qty":
40, "confidence": 0.5}, {"id": "F", "qty": 20, "confidence": 0.5}]}, {"target_items": [{"id": "C", "qty": 5}], "recommen
dations": [{"id": "G", "qty": 30, "confidence": 1.0}]}], "errorCode": 0, "errorMsg": "\u6210\u529f"}

=== slot_match ===
{"assignments": [{"itemId": "1001", "slotId": "S001", "score": 0.961587, "placedCenter": {"x": 5, "y": 3, "z": 1.7}}, {"
itemId": "1002", "slotId": "S003", "score": 0.56881, "placedCenter": {"x": 2, "y": 8, "z": 1.0}}, {"itemId": "1003", "sl
otId": "S001", "score": 0.527815, "placedCenter": {"x": 5, "y": 3, "z": 2.2}}], "errorCode": 0, "errorMsg": "\u6210\u529
f"}

=== importance_classify ===
{"results": [{"itemId": "10001", "itemName": "mask", "score": 1.0, "level": "A"}, {"itemId": "10002", "itemName": "water
", "score": 0.0, "level": "C"}, {"itemId": "10003", "itemName": "cloths", "score": 0.384802, "level": "C"}], "errorCode"
: 0, "errorMsg": "\u6210\u529f"}

C:\Users\ZZS\source\repos\pybind11_工程版\x64\Debug\pybind11_工程版.exe (进程 7500)已退出，代码为 0 (0x0)。
按任意键关闭此窗口. . .
```